Can a Structural Honesty Linter Catch Real Vulnerabilities? A CVE Retrospective Across Three Tools

Bilal Syed Arfeen
BayyinahEnterprise

November 2026

Abstract

We test whether furqan-lint v2.1.0, a structural honesty linter, can flag the code patterns that caused 15 known Python CVEs. We reconstruct each CVE's vulnerable pattern and scan it with furqan-lint, bandit 1.9.4, and ruff 0.16.9 (full rule set). For each tool, we grade whether it flags the root-cause pattern (HIT), flags something else at the location (PARTIAL), or produces no relevant finding (MISS). furqan-lint flags the root cause in 14 of 15 CVEs (93%), bandit in 11 (73%), ruff in 4 (27%). In 5 CVEs, furqan-lint's root-cause finding has no counterpart in either other tool: hasattr masking enabling a getattr injection chain (CVE-2021-23727), a swallowed SSLError enabling a TLS bypass (CVE-2023-40217), a temp-file fd leak without cleanup (CVE-2023-6597), disabled SSL verification (CVE-2024-35195), and hasattr on attacker-controlled config fields (CVE-2026-41523). 1 CVE (CVE-2022-28346, Django ORM kwargs injection) is missed by all three tools. The patterns are reconstructions, not verbatim vulnerable code, and the CVEs were selected by searching for patterns matching furqan-lint rule families, introducing selection bias. The 15 pattern files and all tool outputs are published for independent verification.

---

1. Introduction

Papers 1 and 2 in this series established that furqan-lint produces findings at locations where bandit and ruff do not, and that its novel rule families -- error handling (MR-*), resource lifecycle (RC-*), hasattr masking (SI-007), and interface honesty (SI-005) -- account for most of the non-overlapping signal. Neither paper answered the practitioner's question: *so what?* A finding at a location no other tool flags is only useful if the flagged pattern actually causes harm in production.

This paper tests whether furqan-lint's rules fire on code patterns that led to real, assigned CVEs. We ask: for each CVE, does furqan-lint flag the root cause? Does bandit? Does ruff? And where furqan-lint fires and the others do not, is the root-cause finding genuinely different, or is it detecting the same symptom under a different rule name?

---

2. Method

2.1 CVE selection

We searched for Python CVEs with root causes that map to furqan-lint rule families:

- **Error handling (MR rules):** CVEs where except:pass, broad except, or swallowed typed exceptions caused or enabled the vulnerability
- **Resource lifecycle (RC rules):** CVEs where file descriptors, connections, or temporary files were not properly closed
- **Interface honesty (SI rules):** CVEs where hasattr, assert, or kwargs forwarding enabled the attack
- **Trust boundary (TB rules):** CVEs involving eval, yaml.load, hardcoded secrets, shell injection, or disabled SSL verification

This selection method biases the sample toward CVEs that furqan-lint is designed to catch. A random sample of Python CVEs would produce lower hit rates for all tools. The bias is acknowledged and the selection criteria are published.

We excluded CVEs whose root cause is purely in C extensions, protocol-level flaws, or logic errors that no pattern-matching linter could detect.

2.2 Pattern reconstruction

For each CVE, we wrote a Python file containing a simplified reconstruction of the vulnerable pattern described in the advisory, plus a fixed version. Each file cites the CVE, describes the root cause, and identifies which furqan-lint rule (if any) maps to the vulnerability.

The patterns are reconstructions, not verbatim copies of the original vulnerable code. The actual vulnerable functions may contain additional context (surrounding code, import structures, class hierarchies) that affects tool behavior. We chose reconstruction over verbatim code for three reasons: (1) verbatim code often requires the full project to parse, (2) the patterns are more readable in isolation, and (3) some advisories do not publish the vulnerable code.

2.3 Tool configuration

Tool	Version	Configuration
furqan-lint	v2.1.0	Default (39 active Python rules)
bandit	1.9.4	Default (all rules)
ruff	0.16.9	Default (full rule set, not the 30-rule subset from Paper 2)

2.4 Grading

For each CVE and each tool, we assign one of:

- **HIT**: the tool flags the root-cause pattern or its enabling condition. The finding describes the same class of defect that caused the vulnerability.
- **PARTIAL**: the tool flags the same location but for a different reason. The finding would not lead a developer to the root cause.
- **MISS**: the tool produces no relevant finding on the vulnerable code.

Grading is judgment-based and disclosed in full. Reasonable reviewers may disagree on individual CVEs (see Section 4.4).

---

3. Results

3.1 Aggregate comparison

Metric	furqan-lint	bandit	ruff
HIT (root cause)	14/15 (93%)	11/15 (73%)	4/15 (27%)
PARTIAL	0/15	3/15	2/15
MISS	1/15	1/15	9/15
Unique HIT	4/15	0/15	0/15

3.2 Per-CVE results

#	CVE	CVSS	Root cause	furqan	bandit	ruff
1	2017-1000433	8.1	assert as auth check	HIT SI-006	HIT B101	MISS
2	2018-1000805	--	swallowed auth exception	HIT MR-005	HIT B110	HIT S110
3	2020-14343	9.8	yaml.load unsafe	HIT TB-006	HIT B506	MISS
4	2020-27619	9.8	eval on HTTP content	HIT TB-005	HIT B307	MISS
5	2021-23727	--	hasattr/getattr chain	HIT SI-007	PARTIAL B110	PARTIAL S110
6	2021-38305	7.5	eval on schema input	HIT TB-005	HIT B307	MISS
7	2022-28346	9.8	ORM kwargs injection	MISS	MISS	MISS
8	2022-39227	--	swallowed auth exception	HIT MR-005	HIT B110	HIT S110
9	2023-40217	5.3	swallowed SSLError	HIT MR-005	PARTIAL B504	MISS
10	2023-43791	9.8	hardcoded SECRET_KEY	HIT TB-010	HIT B105	MISS
11	2023-6597	7.4	temp fd leak	HIT RC-005	PARTIAL B110	PARTIAL S110
12	2024-3126	--	shell=True injection	HIT TB-015	HIT B602	MISS
13	2024-35195	5.6	verify=False reuse	HIT TB-022	MISS	MISS
14	2024-45231	5.3	swallowed email error	HIT MR-005	HIT B110	HIT S110
15	2026-41523	7.5	assert + hasattr bypass	HIT SI-006/007	HIT B101	MISS

3.3 The 5 unique catches

In 5 CVEs, furqan-lint's root-cause finding has no counterpart in either other tool:

CVE-2021-23727 (Celery RCE, CVSS unknown): SI-007 flags hasattr(cls, '__call__'), the check that feeds the unrestricted getattr chain. Bandit and ruff flag only the except:pass wrapper -- a symptom, not the injection vector. An engineer following bandit's advice (add logging to the except block) would not discover or fix the getattr chain.

CVE-2023-40217 (CPython TLS bypass, CVSS 5.3): MR-005 flags the empty except block for ssl.SSLError -- the code that swallowed the handshake failure and returned unencrypted data as TLS-protected. Bandit flags ssl.wrap_socket for using the default protocol version (B504), which is unrelated to the vulnerability. Ruff produces no security-relevant finding.

CVE-2023-6597 / Issue 21058 (tempfile fd leak, CVSS 7.4): RC-005 flags mkstemp() without a context manager or cleanup -- the pattern that leaked file descriptors when fdopen() raised. Bandit and ruff flag the except:pass in a loop variant but not the fd leak itself.

CVE-2024-35195 (requests cert bypass, CVSS 5.6): TB-022 flags verify=False, the call that creates the poisoned connection pool entry. Neither bandit nor ruff has any finding on this code.

CVE-2026-41523 (vLLM code execution, CVSS 7.5): SI-006 flags the assert (overlap with bandit B101), but SI-007 also flags both hasattr() calls on attacker-controlled config fields -- the code path that reads untrusted activation function names. Ruff misses entirely.

3.4 The overlaps

In 6 CVEs, all tools that produce a finding agree on the pattern class:

- 3 CVEs (2018-1000805, 2022-39227, 2024-45231): all three tools flag try-except-pass
- 3 CVEs (2020-14343, 2020-27619, 2021-38305): furqan-lint and bandit flag the same dangerous function (yaml.load, eval)

3.5 The miss

CVE-2022-28346 (Django SQL injection, CVSS 9.8): All three tools miss this. The root cause is user-controlled dictionary keys passed via **kwargs to Django ORM methods (annotate, aggregate, extra), which become unsanitized column aliases. furqan-lint's SI-005 detects** kwargs in function definitions, not at call sites. This is a detection gap: no rule currently tracks the flow from user input to ORM kwargs at the call site.

---

4. Discussion

4.1 What the results show

furqan-lint's novel rule families -- the ones identified in Paper 2 as having no bandit/ruff counterpart -- are the same families that produce the unique CVE catches:

Rule family	Paper 2 (unique %)	CVEs uniquely caught	Pattern
SI-007 (hasattr)	88.8%	2021-23727, 2026-41523	Attribute masking in trust boundary
MR-005 (empty except)	86.0%	2023-40217	Swallowed typed exception
RC-005 (temp file)	97.5%	2023-6597	Resource lifecycle gap
TB-022 (verify=False)	--	2024-35195	SSL verification disabled

The overlap decomposition predicted which rules would matter, and the CVE retrospective confirms it on independent evidence.

4.2 What the results do not show

1. **Not a detection rate.** 93% is the hit rate on a biased sample. A random sample of Python CVEs would include logic errors, protocol flaws, and C-extension bugs that no pattern linter can catch.
2. **Not proof of prevention.** Flagging a pattern does not mean the CVE would have been prevented. The flag must be seen, understood, triaged, and acted on. Many of these findings would appear alongside hundreds of other findings in a large codebase scan.
3. **Not verbatim code.** The patterns are reconstructions. The actual vulnerable code may behave differently under these tools.

4.3 The gap

CVE-2022-28346 reveals a real limitation. SI-005 flags **kwargs in function signatures -- "this function accepts arbitrary keyword arguments" -- but not** kwargs at call sites -- "this ORM call receives a user-controlled dictionary." Detecting the latter requires data-flow analysis (tracking where the dict came from), which furqan-lint

does not perform. This is a known class of limitation for pattern-matching linters.

4.4 Grading disputes

Two CVEs have debatable grades:

CVE-2023-40217: We grade bandit as PARTIAL because B504 flags the wrong issue (protocol version, not the swallowed SSLError). A reviewer who considers any ssl-related finding a HIT would score bandit higher.

CVE-2026-41523: We grade furqan-lint's SI-007 as a unique signal (bandit only catches the assert, not the hasattr). A reviewer who considers the assert the primary root cause would not count SI-007 as unique.

---

5. Conclusion

Of 15 Python CVEs, furqan-lint flags the root-cause pattern in 14 (93%), with 5 CVEs where the root-cause finding has no counterpart in bandit or ruff. The unique catches come from the same rule families that Paper 2 identified as genuinely novel: hasattr masking (SI-007), swallowed typed exceptions (MR-005), resource lifecycle (RC-005), and SSL verification (TB-022).

The practical implication: adding furqan-lint alongside bandit in a CI pipeline would have flagged the enabling pattern in 5 additional CVEs that bandit alone missed or misidentified. Whether those flags would have been acted on is a question this study cannot answer.

The honest limitation: the CVE sample is biased toward patterns furqan-lint is designed to catch. The 93% rate does not generalize to arbitrary Python CVEs. The 1 CVE all tools miss (ORM kwargs injection) identifies a real detection gap that requires data-flow analysis beyond pattern matching.

Future work:

- Random-sample CVE study (unbiased hit-rate measurement)
- Verbatim code testing (clone vulnerable versions, scan full repos)
- Developer study: do engineers act on furqan-lint findings?
- New rule: call-site kwargs tracking for ORM injection (CVE-2022-28346 gap)

Data availability

All 15 CVE pattern files, tool outputs, and the grading matrix are in `furqan-lint/paper/cve_retrospective/` in the repository. Tools: furqan-lint v2.1.0, bandit 1.9.4, ruff 0.16.9, Python 3.13.18.

Repository: https://github.com/BayyinahEnterprise/Kun-Fayakun

---

References

[1] B. S. Arfeen, "furqan-lint vs. bandit: A Finding-Count Comparison Across 25 Python Codebases," BayyinahEnterprise, 2026.

[2] B. S. Arfeen, "Three-Tool Overlap Decomposition of furqan-lint Findings Across 25 Python Codebases," BayyinahEnterprise, 2026.

[3] PyCQA, "Bandit: Security linter for Python," v1.9.4, https://bandit.readthedocs.io/

[4] Astral Software, "Ruff: An extremely fast Python linter," v0.16.9, https://docs.astral.sh/ruff/

[5] MITRE, "Common Vulnerabilities and Exposures," https://cve.mitre.org/

Conflict of interest. The author develops furqan-lint. Analysis scripts were written in a Claude (Anthropic) sandbox.
